

How Quantum Computers can break RSA, and a Presentation of Quantum-Resistant Cryptographic Schemes

Cas Bomans, Ehsan Goodarzi, Mika Müller,
Malte Opderbeck, and Emil Schläger

May 2025

Abstract

RSA is a widely used encryption algorithm that secures data by relying on the difficulty of factoring large numbers. It is part of a lot of the security measurements of today's internet communication. However, quantum computers, using Shor's algorithm, can factor these numbers efficiently, potentially breaking RSA encryption and exposing sensitive data — making current cryptographic systems vulnerable in a future with practical quantum computing.

Shor's algorithm is the quantum algorithm that poses this threat. By using quantum superposition, interference, and the Quantum Fourier Transform, it can efficiently find the period of modular exponentiation — a crucial step in factoring large numbers. This section explains how Shor's algorithm works, step by step, and why it represents a fundamental shift in the security assumptions behind RSA.

To demonstrate the real-world threat Shor's algorithm poses to RSA, we implemented its quantum part using Qiskit. Our code simulates the quantum circuit, applies the Quantum Fourier Transform, and estimates the period of a modular function—mimicking how a quantum computer would factor integers. This hands-on demo connects theory with practical quantum programming.

Kyber is a quantum resistant key encapsulation mechanism; it establishes a shared secret key between two communicating parties. It is based on the module learning with errors problem, which is believed to be computationally hard to solve, even for quantum computers. Because of its performance, it was chosen to be standardized under the name ML-KEM.

Lastly, we will present SLH-DSA, also known as SPHINCS+, as a quantum-resistant digital signature scheme. SLH-DSA bases its quantum-resistance on the pre-image resistance property of hash functions, and improves upon other hash-based schemes like XMSS by removing the need for statefulness. In combination, this allows SLH-DSA to securely sign up to 2^{64} messages.

1 Cybersecurity and the quantum threat

This chapter provides an initial perspective on the intersection of cybersecurity and quantum computing, focusing on the vulnerabilities that arise from quantum advancements. Given the breadth and complexity of cybersecurity protocols and concepts that protect modern digital communications, a comprehensive treatment is beyond the scope of this text. Instead, we narrow the focus to one of the most widely deployed cryptographic algorithms: the RSA algorithm.

We begin by examining RSA’s role in securing contemporary online infrastructures, showing its functional principles, intended security guarantees, and the types of problems it addresses. This foundation allows us to critically assess where RSA becomes susceptible to attacks enabled by quantum computing. Specifically, the chapter explores how Shor’s algorithm—a quantum algorithm capable of efficiently factoring large integers—poses an existential threat to RSA-based cryptosystems.

The subsequent sections offer both a theoretical exposition and a practical demonstration of how Shor’s algorithm compromises RSA. Finally, the chapter concludes by surveying two emerging cryptographic approaches designed to withstand quantum attacks, pointing the way toward a more resilient future in post-quantum cybersecurity.

1.1 The RSA algorithm

The RSA algorithm—named after its inventors Ron Rivest, Adi Shamir, and Leonard Adleman—was first published in 1977 [1] and quickly became a cornerstone of modern cryptography. As one of the earliest and most influential public-key cryptosystems, RSA introduced a paradigm shift in secure digital communication by enabling asymmetric encryption: a method where encryption and decryption are performed using different, mathematically related keys. This innovation addressed a fundamental challenge in cryptography—secure key exchange over untrusted channels. As Figure 1 shows, Bob can generate a key pair (k_{pub}, k_{pr}) and send the public key to Alice. Alice can on her turn encrypt a message using Bob’s public key, and Bob can decrypt this message using the private key he generated.

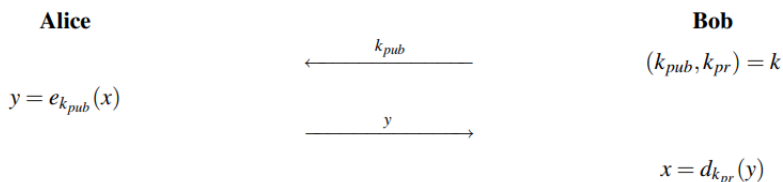


Figure 1: Asymmetrical Encryption [2]

At the time of its introduction, RSA represented a significant advancement

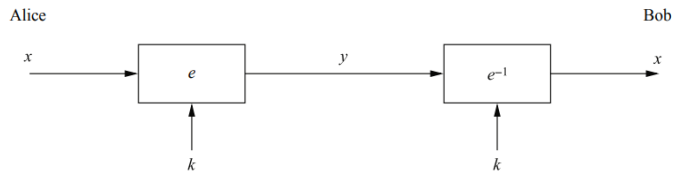


Figure 2: Symmetric Encryption [2]

over symmetric-key systems, which required both parties to share a secret key in advance (see Figure 2), often through impractical or insecure means. The asymmetry of RSA made it particularly suitable for the emerging internet infrastructure, where secure communication between previously unknown parties became essential.

Today, RSA remains deeply embedded in digital security protocols. It is widely used for establishing secure connections via the Transport Layer Security (TLS) protocol, particularly in the authentication phase where digital certificates often rely on RSA-based signatures [3]. Although newer key exchange mechanisms, such as Elliptic Curve Diffie-Hellman (ECDHE), are increasingly favored for their performance and forward secrecy benefits, RSA is still prevalent in certificate authorities and server authentication. Recent telemetry data indicates that over 50 percent of websites continue to accept RSA certificates for identity verification during TLS handshakes [3].

Beyond web traffic, RSA underpins the security of email encryption standards like S/MIME and PGP, secures data in virtual private networks (VPNs), and is commonly used in secure shell (SSH) access for remote system administration. It also plays a role in the security of software updates and digital signatures, ensuring the integrity and authenticity of the transmitted code.

1.2 RSA for Encryption and Digital Signing

As explained above, RSA serves two primary purposes in securing digital communication: encryption and digital signing. In RSA encryption, the sender uses the recipient’s public key to encrypt a message, ensuring that only the recipient, who holds the corresponding private key, can decrypt it and retrieve the original message. The encryption process begins by representing the plaintext message M as an integer, which must be smaller than the modulus n . The public key of the recipient consists of two components: e , the public exponent, and n , the modulus, which is the product of two large prime numbers. The sender applies the recipient’s public key to the message using modular exponentiation:

$$C = M^e \mod n$$

where C represents the ciphertext, the encrypted version of the message. The process ensures that even if the ciphertext is intercepted by an attacker, it cannot

be decrypted without access to the private key. The recipient can decrypt the ciphertext using their private key d by performing the inverse operation:

$$M = C^d \mod n$$

In addition to encryption, RSA is also used for digital signing, which ensures both the authenticity and integrity of a message. When the sender wishes to digitally sign a message, they use their private key to generate a unique signature for that message. The signature acts as proof that the message was indeed sent by the holder of the private key and that the message has not been altered during transmission. To create the signature, the sender applies their private key d to the message M using modular exponentiation:

$$S = M^d \mod n$$

The resulting signature S is then sent along with the message. The recipient, upon receiving the signed message, can verify its authenticity by using the sender's public key. The verification process involves applying the sender's public key e to the signature S and checking if it matches the original message M . The recipient computes:

$$M' = S^e \mod n$$

If M' matches the original message M , it confirms that the signature is valid. This ensures that the message was indeed signed by the sender, and it has not been modified after signing. RSA digital signatures provide a way to guarantee that messages are both authentic (i.e., they came from the claimed sender) and intact (i.e., they have not been altered during transmission).

1.3 How is an RSA key-pair generated?

The RSA algorithm relies on the mathematical properties of large prime numbers and modular arithmetic to generate a secure public-private key pair. The security of RSA is based on the practical difficulty of factoring large composite numbers, a problem for which no efficient classical algorithm is known. The following section will walk through the process of generating an RSA key pair as described by Christof Paar[2].

1.3.1 Choose Two Large Prime Numbers

Select two distinct large prime numbers, denoted as:

$$p \quad \text{and} \quad q$$

These primes should be chosen randomly and independently of each other, and they should be of similar bit-length to ensure security.

1.3.2 Compute the Modulus

Compute the modulus n , which will be used as part of both the public and private keys:

$$n = p \cdot q$$

The modulus n is a crucial component in both encryption and decryption operations.

1.3.3 Compute Euler's Totient Function

Calculate Euler's totient function $\varphi(n)$, which for the product of two primes is given by:

$$\varphi(n) = (p - 1)(q - 1)$$

This function represents the number of integers less than n that are coprime to n .

1.3.4 Choose the Public Exponent

Select an integer e such that:

$$1 < e < \varphi(n) \quad \text{and} \quad \gcd(e, \varphi(n)) = 1$$

The value e is typically chosen to be a small odd integer, commonly $e = 65537$, for efficiency and security reasons.

1.3.5 Compute the Private Exponent

Compute the private exponent d , which is the modular multiplicative inverse of e modulo $\varphi(n)$. This means:

$$d \cdot e \equiv 1 \pmod{\varphi(n)}$$

The value d satisfies:

$$d = e^{-1} \pmod{\varphi(n)}$$

1.3.6 Output the Key Pair

The public key consists of the pair:

$$(e, n)$$

The private key consists of the pair:

$$(d, n)$$

The public key is used for encryption and digital signature verification, while the private key is used for decryption and signing.

This key generation process forms the basis of RSA's security model, assuming that the prime factors p and q remain secret and that factoring n is computationally infeasible.

1.4 Classic Process of Calculating the Private Key

Given the RSA public key $k_{pub} = (e, n)$, the goal of an attacker is to compute the private key $k_{pr} = (d, n)$ using the public information n and e . This process is based on the fact that the private key d is the modular multiplicative inverse of e modulo Euler's totient function $\varphi(n)$.

1.4.1 Key Relationship

Recall that the private key d is computed such that:

$$d \cdot e \equiv 1 \pmod{\varphi(n)}$$

This means that d is the multiplicative inverse of e modulo $\varphi(n)$. The challenge is that, in a standard RSA setup, $\varphi(n)$ is unknown because it depends on the factors of n , specifically p and q , the two large prime numbers used in the generation of n . Therefore, the attacker's first task is to recover $\varphi(n)$.

1.4.2 Factoring

n To compute $\varphi(n)$, the attacker must factor n into its two prime factors, p and q , since:

$$\varphi(n) = (p-1)(q-1)$$

Once the prime factors p and q are determined, the attacker can compute $\varphi(n)$, and subsequently, find the value of d by solving the equation:

$$d = e^{-1} \pmod{\varphi(n)}$$

1.4.3 Why This Method is Not Feasible with Current Classical Computers

Factoring a large integer n into its prime factors is considered a computationally hard problem for classical computers, particularly when n is sufficiently large, such as the typical sizes used in RSA encryption (e.g., 2048-bit or 4096-bit numbers). The best-known classical algorithms for factoring, such as the general number field sieve (GNFS), have exponential time complexity with respect to the number of bits in n . For an RSA modulus of 2048 bits, for instance, factoring n using GNFS would take billions of years with current technology.

The difficulty of factoring large numbers is the cornerstone of RSA's security, as there is no known efficient classical algorithm that can factor a number of this size in a reasonable amount of time. This makes the classic process of computing the private key infeasible with today's classical computing power.

However, this assumption changes in the presence of quantum computing, where Shor's algorithm provides an exponentially faster method for factoring large integers. This algorithm poses a significant threat to RSA encryption, as it could allow an attacker to recover the private key d in polynomial time, rather than the exponential time required by classical algorithms.

2 Shor's Algorithm

2.1 Introduction

Classical cryptographic methods such as RSA are based on the difficulty of factorizing large integers, a task for which no efficient classical algorithm is known [4], [5]. Even the best classical methods, such as the number field sieve, require sub-exponential time, making RSA with 2048-bit keys practically unbreakable [4].

In 1994, Peter Shor demonstrated that quantum computers could, in theory, solve the factoring problem in polynomial time [6]. His algorithm uses quantum principles such as superposition and interference, which threatens the long-term security of RSA once quantum computers are operational on a large scale [4], [5].

2.2 The Core Idea of Shor's Algorithm

The central idea of Shor's algorithm is to reduce the problem of prime factorization to a problem of finding a period.

2.2.1 From factoring to period finding

The goal is to factor a large composite number N , that is, to determine its nontrivial prime factors. The algorithm begins by randomly selecting an integer a , such that $1 < a < N$ and $\gcd(a, N) = 1$ [4], [5]. Based on this value, the following function is defined:

$$f(x) = a^x \bmod N$$

This function is periodic, which means that there exists a smallest positive integer r such that:

$$f(x + r) = f(x)$$

The value r is called the *period* of the function.

2.2.2 Why the period helps

If the period r is even and $a^{r/2} \not\equiv -1 \bmod N$, then with high probability, nontrivial factors of N can be computed using:

$$\gcd(a^{r/2} \pm 1, N)$$

Knowing the period therefore makes it possible to efficiently solve the original factoring problem [4], [6].

2.2.3 Summary

Shor's algorithm reduces the problem of factoring to the task of finding the period of a modular exponential function. The algorithm can be divided into three main steps:

1. Select a random integer a such that $1 < a < N$ and $\gcd(a, N) = 1$,
2. Determine the period r of the function $f(x) = a^x \bmod N$, where $f(x+r) = f(x)$,
3. Compute the factors of N using $\gcd(a^{r/2} \pm 1, N)$, assuming r is even and $a^{r/2} \not\equiv -1 \bmod N$.

Although steps 1 and 3 can be executed efficiently on a classical computer, the real challenge lies in step 2: *finding the period r* . This is exactly where the quantum computer demonstrates its advantage over classical methods.

2.3 Quantum Part: Finding the Period

2.3.1 Introduction: Why use a quantum computer at all?

Classical methods provide only highly inefficient approaches to finding the period r , either through brute-force search or mathematically complex procedures. A quantum computer, on the other hand, can exploit fundamental quantum-mechanical principles - such as superposition and interference - to extract global properties of a function, such as its period.

The goal is to determine the period r of the function $f(x) = a^x \bmod N$ without having to evaluate $f(x)$ for every value of x individually [4].

2.3.2 State preparation and modular exponentiation

The quantum period-finding algorithm begins by initializing two quantum registers. The first register contains n qubits and will hold the superposition of input values x . The second register is used to compute the function value $f(x) = a^x \bmod N$.

Initially, both registers are in the state:

$$|0\rangle^{\otimes n} \otimes |0\rangle$$

A Hadamard transform is applied to each qubit in the first register, resulting in a uniform superposition of all values x :

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle \otimes |0\rangle$$

where $Q = 2^n$ is a power of two such that $Q > N^2$ [4].

Next, a quantum circuit is used to compute the modular exponentiation $f(x) = a^x \bmod N$ in the second register. The overall state becomes:

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle \otimes |f(x)\rangle$$

This step is essential, as it encodes the periodic structure of the function into the quantum state [4], [6].

2.3.3 Measurement of the second register

After computing the modular exponentiation, the quantum state is:

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle \otimes |f(x)\rangle$$

At this point, a measurement is performed on the second register. This collapses it to a specific function value $f(x_0)$, where x_0 is some unknown input. Due to quantum measurement, the first register is projected onto a superposition of all x values that satisfy:

$$f(x) = f(x_0)$$

Because $f(x) = a^x \bmod N$ is periodic with period r , the remaining state in the first register is a uniform superposition over all integers of the form:

$$x_0 + j \cdot r \quad \text{for } j = 0, 1, \dots, m-1$$

This gives:

$$\frac{1}{\sqrt{m}} \sum_{j=0}^{m-1} |x_0 + j \cdot r\rangle$$

where $m \approx Q/r$ is the number of such values in the domain. This state contains the hidden periodicity and is the input to the Quantum Fourier Transform (QFT) [4], [5].

2.3.4 Quantum Fourier Transform (QFT)

The Quantum Fourier Transform (QFT) is applied to the first register in order to extract information about the hidden period r encoded in the superposition:

$$\frac{1}{\sqrt{m}} \sum_{j=0}^{m-1} |x_0 + j \cdot r\rangle$$

Mathematically, the QFT maps a computational basis state $|x\rangle$ to a new superposition of basis states $|k\rangle$ as follows:

$$|x\rangle \mapsto \frac{1}{\sqrt{Q}} \sum_{k=0}^{Q-1} e^{2\pi i x k / Q} |k\rangle$$

Applying this transformation to the periodic state results in constructive interference at basis states $|k\rangle$ where $k/Q \approx s/r$ for some integer s . In other words, the QFT amplifies the probability amplitudes for values of k that are close to integer multiples of Q/r [4].

The QFT can be implemented efficiently using quantum gates, with a complexity of $\mathcal{O}(n^2)$ for n qubits. This makes it feasible for use in quantum algorithms such as Shor's [4], [5].

2.3.5 Measurement of the first register

After applying the Quantum Fourier Transform (QFT), the first register is in a superposition of basis states where constructive interference has occurred. These are the states $|k\rangle$ for which the measurement outcome k satisfies:

$$\frac{k}{Q} \approx \frac{s}{r}$$

for some integer s , where r is the unknown period and Q is the size of the computational domain (typically $Q = 2^n$). The measurement of the first register results with high probability in a value of k that is close to one of these fractional multiples.

2.3.6 Classical postprocessing: Recovering the period

The final step is performed classically: using the measured value k , we approximate the ratio s/r by reducing the fraction k/Q via a continued fraction expansion [4]. This results in a candidate for r .

To validate whether the recovered r is correct, we check whether it satisfies the condition:

$$a^r \bmod N \equiv 1$$

If the candidate value does not fulfill this condition, the algorithm is repeated with a new random selection of a . Statistically, after a small number of repetitions, the procedure results in a correct period with a high probability. [4], [5].

2.3.7 Summary

The quantum part of Shor's algorithm is responsible for transforming the classical factoring problem into a period-finding problem that can be solved efficiently using quantum computation. The process utilizes:

- Quantum parallelism to evaluate all $f(x)$ simultaneously in superposition,
- Interference and the Quantum Fourier Transform to highlight the period,
- A single measurement to extract information about the hidden frequency,
- Classical postprocessing (continued fractions) to recover the period r .

This quantum subroutine runs in polynomial time and provides the exponential speedup that distinguishes Shor's algorithm from all known classical factoring methods.

3 Classical Postprocessing

After the quantum part of Shor's algorithm has produced a candidate for the period r of the function $f(x) = a^x \bmod N$, the remaining steps are entirely classical and relatively efficient. The goal is now to use r to compute a non-trivial factor of the composite number N .

3.1 Verifying the period

To ensure that the period r obtained from the quantum procedure is correct, we verify the condition:

$$a^r \bmod N = 1$$

If this congruence does not hold, the recovered r is incorrect, and the algorithm must be repeated with a new random choice of a [4].

3.2 Computing the greatest common divisor

If r is even and $a^{r/2} \not\equiv -1 \bmod N$, we proceed with the key step: computing

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N)$$

At least one of these expressions results in a non-trivial factor of N [4], [5]. This relies on the identity:

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1)$$

and the fact that N divides $a^r - 1$, but not necessarily each of the two factors. If $a^{r/2} \equiv \pm 1 \bmod N$, then the algorithm must be restarted with a new base a .

3.3 Why it works

The mathematical justification stems from number theory. The relation $a^r \equiv 1 \bmod N$ implies that a is an element of the multiplicative group modulo N , and r is its order. Under the condition that r is even and $a^{r/2} \not\equiv \pm 1 \bmod N$, the values $a^{r/2} - 1$ and $a^{r/2} + 1$ are non-trivial divisors of N in most cases.

This method succeeds with high probability if N is the product of two distinct large primes, as is the case for RSA. If the conditions are not met, the algorithm is repeated, typically requiring only a few iterations.

3.4 Summary

The classical postprocessing phase is responsible for translating the period r into an actual factorization of the integer N . It involves:

- Verifying the correctness of the period via modular exponentiation,
- Checking whether r is even and $a^{r/2} \not\equiv \pm 1 \pmod{N}$,
- Computing the greatest common divisors $\gcd(a^{r/2} \pm 1, N)$,
- Repeating the process if necessary, with a different choice of a .

These steps are all efficient and run in polynomial time on a classical computer. The overall complexity of Shor’s algorithm remains polynomial when combining the quantum and classical phases [4]–[6].

4 Limits and Why the Algorithm Still Plays a Role

Although Shor’s algorithm provides an exponential speedup for integer factorization, it is not yet practical for attacking cryptographic schemes like RSA due to current hardware limitations.

Technological barriers

Executing Shor’s algorithm for large numbers (e.g., RSA-2048) would require thousands of logical qubits and millions of physical qubits with reliable error correction [7]. Present-day quantum computers are far from meeting these requirements, mainly due to decoherence, gate errors, and limited qubit counts [4].

Why it still matters

Despite these limitations, Shor’s algorithm is a milestone in quantum computing. It shows that quantum computers can solve certain problems exponentially faster than classical machines. Its discovery has accelerated research in quantum hardware and post-quantum cryptography [5], [6].

Even though real attacks on RSA are not yet possible, the threat of quantum computers becoming powerful enough in the future is severe enough to require migration to post-quantum cryptographic schemes as fast as possible. This is due to a practice called Store-Now-Decrypt-Later, the practice of intercepting messages now, and storing them on hard drives in hopes of cracking them once quantum computers have become strong enough. This is especially a risk in national security applications, where highly confidential information might still be relevant decades later.

5 Shor's Algorithm Implementation in Qiskit: A Quantum Simulation Approach

5.1 Implementation in Qiskit

In the following section, I will present an example implementation of Shor's algorithm with the help of [8] and [9]. The implementation is done using Qiskit, IBM's framework for developing with quantum computers a

5.2 Controlled Modular Multiplication

To perform the modular exponentiation required in the quantum phase estimation, we use a controlled unitary operation. This operation performs modular multiplication of a quantum register:

```
1
2 class ControlledMultiplier(QuantumCircuit):
3     def __init__(self, base, bit_index, modulus):
4         super().__init__(modulus.bit_length())
5         self.base = base
6         self.power = 2 ** bit_index # Convert binary to decimal
7         self.modulus = modulus
8         self.name = f'{self.base}^{self.power} mod {self.modulus}'
9         self._create_circuit()
10
11     def _create_circuit(self):
12
13         for dec_power in range(self.power):
14             mod_result = self.base ** dec_power % self.modulus
15             for i in range(self.num_qubits):
16                 if mod_result >> i & 1: self.x(i)
17                 for j in range(i + 1, self.num_qubits):
18                     if mod_result >> j & 1: self.swap(i, j)
```

This circuit constructs a modular multiplication $a^x \bmod N$, which is essential in defining the unitary operation

$$U|x\rangle = |a \cdot x \bmod N\rangle$$

The operation is repeated under exponentiation to encode periodicity into quantum states.

5.3 Quantum Phase Estimation (QPE)

Quantum Phase Estimation is at the core of the period-finding process. It uses a register of counting qubits initialized with Hadamard gates, applies a sequence

of controlled-unitary operations, and finishes with the inverse Quantum Fourier Transform (QFT):

```

1
2
3 class QuantumPhaseEstimator(QuantumCircuit):
4     # Hadamards, controlled-unitary, and inverse QFT
5     def __init__(self, base, modulus):
6
7     def _apply_controlled_power(self):
8
9     def _build_qpe_circuit(self):
10
11     def simulate_measurement(self, simulator):
12

```

The QFT, already introduced in the earlier sections of the article, is utilized here to transform the amplitude-encoded periodic information into a measurable phase. The measured output gives an estimate of the ratio s/r , where r is the order of a modulo N .

5.4 Algorithm Controller

This part of the code encapsulates both the classical and quantum logic of Shor’s algorithm. It includes initial checks, execution of the quantum subroutine, and post-processing to derive factors from measurement results.

```

1
2
3 class ShorAlgorithm:
4     def __init__(self, N):
5         self.N = N
6
7     def run(self):
8         # Classical pre-checks
9         # Construct QPE circuits
10        # Execute and interpret measurement outcomes
11

```

The classical steps include selecting a random base a , checking $\gcd(a, N)$, and verifying whether the found period r leads to non-trivial factors. The quantum subroutine integrates modular multiplication and QPE, while the post-processing uses continued fraction expansion to estimate r and compute the greatest common divisors.

5.5 Quantum circuit diagram and example

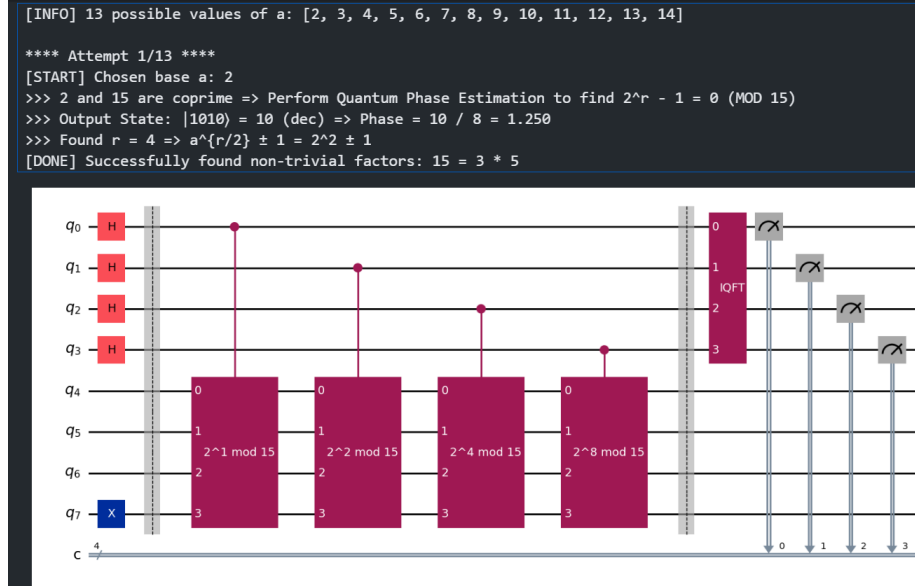


Figure 3: Quantum circuit diagram and example [9]

In our example, the quantum algorithm attempts to factor the number 15 by choosing a random base $a = 2$, which is coprime with 15. Using Quantum Phase Estimation (QPE), it estimates the period r of the function $a^x \bmod N$. The quantum output state $|1010\rangle$ corresponds to the decimal value 10, giving a phase of $10/8 = 1.25$. From this, the period $r = 4$ is determined. Since r is even and $a^{r/2} \pm 1 = 2^2 \pm 1 = 5$ and 3, the algorithm successfully finds the non-trivial factors of 15: 3×5 .

The diagram represents the structure of the quantum part of Shor's algorithm, illustrating the sequence of operations applied to the quantum registers. It begins with initializing the qubits and applying Hadamard gates to create a uniform superposition. Then, an **X(NOT)** gate is applied to the auxiliary qubit to prepare it in the $|1\rangle$ state. This is followed by a controlled modular multiplication subcircuit, which encodes the periodicity of the function $a^x \bmod N$ into the quantum state. Afterward, the inverse Quantum Fourier Transform (IQFT) is applied to the main quantum register to extract the periodicity by interference, which includes Hadamard and controlled phase rotation gates. Finally, the qubits are measured to collapse the state and read out a result that can be used in the classical post-processing step to determine the factors of N .

5.6 Results and Evaluation

Simulation was performed for $N = 15$, where valid values like $a = 2$ and $a = 7$ lead to correct periods (e.g., $r = 4$). Measurement results from the simulator are interpreted using continued fraction expansion. The correct factors (3 and 5) are derived from the calculated r . The implementation is limited by qubit count and noise, hence larger values of N are not practical on current hardware. This simulation demonstrates the successful execution of Shor’s algorithm using Qiskit. By integrating mathematical principles into code, we provide clear evidence of how quantum period-finding leads to integer factorization. While constrained by hardware limitations, the implementation showcases how quantum mechanics enables polynomial-time factorization. As quantum hardware matures, practical demonstrations on larger integers could undermine classical cryptographic assumptions, necessitating the transition to post-quantum security models.

6 ML-KEM

The Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) is an implementation of the CRYSTALS-Kyber algorithm, standardized by the National Institute of Standards and Technology (NIST). CRYSTALS-Kyber is designed to be resistant to attacks by quantum computers.[10]. As a Key-Encapsulation Mechanism (KEM) it establishes a shared secret key between two parties that can then be used as basis for encryption.

6.1 Learning With Errors (LWE)

Learning With Errors (LWE) is a mathematical problem that forms the basis for the security of certain lattice-based cryptographic systems. The security of ML-KEM is ultimately related to the computational difficulty of the LWE problem.

In the LWE problem, the input consists of a set of random *noisy* linear equations involving some secret variables over a finite field $\mathbb{Z}_q$. The goal is to recover the value of these secret variables. The term *noisy* is crucial because each equation includes a small, randomly generated error term. This error is what makes the problem hard; it prevents standard techniques like Gaussian elimination from efficiently solving the system of equations to find the secret [10].

A simple form of the LWE problem can be illustrated as follows: Suppose we are working modulo $q = 89$ which is a prime number and with only 4 dimensions. We choose secret values

$$\begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{pmatrix} 10 \\ 82 \\ 50 \\ 5 \end{pmatrix} \in \mathbb{Z}_q^4$$

We create a set of linear equations based on the first equation where e_i is a small error near zero.

$$a_i \cdot x + b_i \cdot y + c_i \cdot z + d_i \cdot w + e_i \equiv r_i \pmod{q}$$

$$77 \cdot x + 7 \cdot y + 28 \cdot z + 23 \cdot w - 3 \equiv 8 \pmod{89}$$

$$21 \cdot x + 19 \cdot y + 30 \cdot z + 48 \cdot w + 2 \equiv 39 \pmod{89}$$

etc...

The LWE problem is: Given the set of *noisy* equations, find the secret values, with the knowledge that these samples were generated using small error terms e_i . Finding the secret values would be trivial without the error terms or when knowing the exact error values. The small, unpredictable errors make finding the secret values computationally hard, especially with a large number of variables or high dimensions. LWE is considered hard to solve even for adversaries with a quantum computer [11].

6.2 Module Learning With Errors (MLWE)

Module Learning With Errors is a generalization of Learning With Errors. While LWE uses integers modulo q ($\mathbb{Z}_q^n$), MLWE replaces $\mathbb{Z}_q^n$ with a certain module R_q^k which is the k -fold Cartesian product of a certain polynomial ring R_q .

In MLWE the secret is an element of the Module R_q^k [10].

ML-KEM uses the number-theoretic transform (NTT) which enables faster calculations with MLWE [10].

6.3 Component scheme K-PKE

K-PKE is a public-key encryption scheme that serves as a component of the ML-KEM scheme. It is not approved for use as a stand-alone cryptographic scheme [10]. K-PKE is based on the MLWE problem.

K-PKE consists of the three algorithms: Key Generation, Encryption and Decryption [10].

6.3.1 Key Generation

The Key generation algorithm takes randomness as input and outputs an encryption key and a corresponding decryption key [10]. The encryption key can be made public, while the decryption key must remain private.

Informally, the decryption key is a vector s of elements from the ring R_q , while the encryption key is a collection of "noisy" linear equations ($A, As + e$) using the secret variables s and the "noise" e .

The matrix A is generated pseudorandomly, and s and e are sampled from centered binomial distributions. Note: Only the seed used to generate the matrix A is transmitted. This allows others to regenerate it.

6.3.2 Encryption

The encryption algorithm takes an encryption key, a plaintext message (size fixed to 32 bytes) and randomness as input and outputs a ciphertext [10].

Informally, it is possible to combine equations from the matrix A to form new valid equations. One could encode a single bit by sending either the correct equation or a significantly altered equation. In the case of K-PKE, the term is a polynomial with 256 coefficients, so one could encode one bit with each coefficient.

6.3.3 Decryption

The decryption algorithm takes a decryption key and a ciphertext as input and outputs the plaintext [10].

Informally, using the decryption key one can calculate the correct result of the equation in the ciphertext. The difference between the corrected solution and the solution in the ciphertext is the transmitted message.

6.4 Fujisaki-Okamoto Transform

The Fujisaki-Okamoto (FO) Transform is used to make K-PKE more secure. K-PKE is only IND-CPA-secure, a simplified explanation is that an attacker is only able to guess between two plaintexts for a known ciphertext. Using the Fujisaki-Okamoto Transform it is also IND-CCA2-secure, a simplified explanation is that an attacker is only able to guess between two plaintexts for a known ciphertext, even when the attacker can ask an orracle for the decryption of any other ciphertext [10].

6.5 Parameter Sets & Implementation Considerations

The National Institute of Standards and Technology defined the following three parameter sets for ML-KEM.

Specification	Encapsulation key [in bytes]	Decapsulation key [in bytes]	ciphertext [in bytes]	Decapsulation failure rate
ML-KEM-512	800	1632	768	$2^{-138.8}$
ML-KEM-768	1184	2400	1088	$2^{-164.8}$
ML-KEM-1024	1568	3168	1568	$2^{-174.8}$

Table 1: Parameter sets and sizes of ML-KEM [10]

The number (512, 768, 1024) does not correspond to an internal bit count, instead it is supposed to be at least as secure as a generic block cipher with the prescribed key size [10].

Decapsulation failures, where the recipient’s calculated shared secret does not match the sender’s, can theoretically occur, but the probability is overwhelmingly low for the approved parameter sets.

7 SLH-DSA

The Stateless Hash-based Digital Signature Algorithm (SLH-DSA) is a digital signature scheme designed to be quantum-resistant. It has been standardized by the American National Institute of Standards and Technology (NIST), after being selected as one of the winners of their 2022 competition. NIST states that SLH-DSA is considered to be a backup, in case "ML-DSA [another winner of the competition, based on the mathematical problem of lattices] proves vulnerable" [12]. In this section, we will present a summary of the fundamental algorithmic schemes that make up SLH-DSA, and provide an explanation as to why it is considered secure even against quantum computers.

7.1 Hash Functions

A Hash Function is any function that projects a binary string of arbitrary many digits into a binary string of fixed length n :

$$H : \{0, 1\}^* \longrightarrow \{0, 1\}^n$$

The image of the function is called a "hash" or a "digest". From a cryptographic point of view, there are a couple of criteria which a Hash Function needs to meet, in order to be considered cryptographically secure (also called a "Cryptographic Hash Function") [13]:

1. **Pre-image Resistance:** Given only the output of a Hash Function $H(m)$, it should be computationally hard to find the input m .
2. **Collision Resistance:** It should be computationally hard to find two different inputs m_1 and m_2 , so that $H(m_1) = H(m_2)$.

An example for a cryptographic Hash Function is the family of SHA-2 algorithms.

7.2 WOTS+

The Winternitz One-Time Signature Scheme (WOTS+) is the centerpiece of SLH-DSA. It is a hash-based signature scheme, which, as its name suggests, can sign exactly one message per key pair. This section will provide a top-level breakdown of the algorithm as presented by [14], and illustrate why the scheme can only be used once securely.

Key Generation The WOTS+ signature scheme configures two parameters: First, the length of the Message in bytes, $n := \frac{|msg|}{8}$, and second, the so-called Winternitz Parameter, w . w can be chosen to be either 4 or 16. These two parameters are used to then generate a third parameter, called len :

$$len_1 := \frac{8n}{ld(w)}$$

$$len := \left\lfloor \frac{\log_2(len_1)(w-1)}{\log_2(w)} + 1 \right\rfloor + len_1$$

With the parameters n , w and len defined, one can generate a private key wsk by obtaining len strings of n randomized bytes. The public key is obtained from this private key, by applying a Cryptographic Hash Function repeatedly to each string wsk_i of the private key:

$$wpk := (H^{w-1}(wsk_1), H^{w-1}(wsk_2), \dots, H^{w-1}(wsk_{len}))$$

Signature Generation and Verification The WOTS+ signature is obtained in a fashion similar to computing the public key. First, the message is split into len_1 chunks $m_{1..len_1}$. If interpreted as a decimal integer $dec(m_i)$, the definition of len_1 ensures that each $dec(m_i) \leq w-1$. To obtain the signature of the message, the Hash function is then applied m_i times to the corresponding string of the private key, sk_i :

$$\sigma := (H^{m_1}(wsk_1), H^{m_2}(wsk_2), \dots, H^{m_{len_1}}(wsk_{len_1}))$$

To verify the signature, the receiver computes

$$wpk'_i := H^{w-1-m_i}(\sigma_i)$$

The signature then is valid, if $wpk'_i = wpk_i$ for every (wpk'_i, wpk_i) .

Forgery If a WOTS+ key pair has previously been used, it is easy for an attacker to forge a signature of a future message without having to know the sender's private key. Assuming $\sigma_i = H^{m_i}(wsk_i)$, an attacker could forge a message where $\hat{m}_i \geq m_i$. An attacker can now easily forge the partial signature $\hat{\sigma}_i$ by applying H $\hat{m}_i - m_i$ times:

$$\hat{\sigma}_i = H^{\hat{m}_i - m_i}(\sigma_i)$$

As a result, a signature that has been obtained with a previously used WOTS+ key pair cannot be trusted.

7.3 XMSS

The Extended Merkle Tree Signature Scheme (XMSS) is a stateful signature scheme combining multiple WOTS+ key pairs in a way so that each XMSS key pair can be used multiple times. It introduces the Merkle Tree structure, a binary tree where each node is obtained by hashing the concatenation of its two children. The tree is computed starting from the leaves, which are a plain-text string. In the case of XMSS, the leaves are each the public key of a unique WOTS+ key pair, hence the "extended" name.

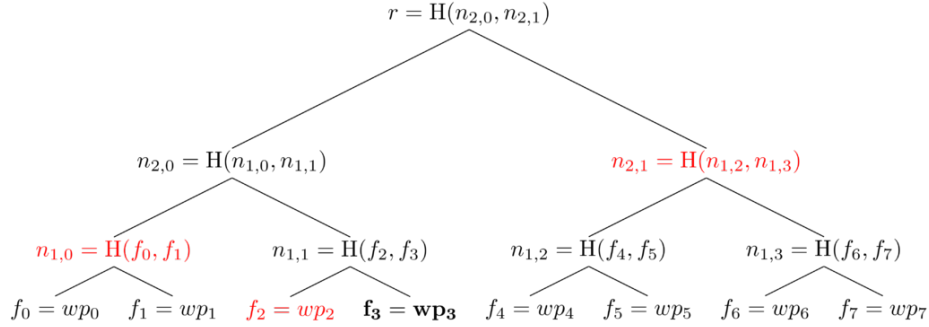


Figure 4: An eight-leaf Merkle Tree [15]

A central property of a Merkle Tree is the so-called **Authentication Path**, indicated in Figure 4 by the nodes coloured red. The authentication path is a set comprising of exactly one node from each layer of the tree, and one additional leaf, selected in such a way that the root of the tree can be re-computed from only those nodes.

Key Generation To generate the series of WOTS+ private keys used as leaves for the Merkle Tree, a Pseudo-random function (PRF) is used. A PRF is a function that generates a series of randomized numbers, starting from a seed s . A PRF is deterministic in the sense that if given the exact same seed, the PRF will produce the exact same series of values every time. The seed used to generate the WOTS+ private keys is the private key of the XMSS. The public key is the root of the Merkle tree.

Signature Generation and Verification In order to generate the XMSS signature of a message, the sender picks a leaf at random. The signature then comprises of:

- The WOTS+ signature of the message
- The Authentication Path including the WOTS+ public key

The signature is then verified by the receiver by first validating the WOTS+ signature, and subsequently re-computing the root of the Merkle Tree using the Authentication Path. If the WOTS+ signature is valid, and the re-computed root is equal to the sender's public key, the signature is valid.

Statefulness As using the same WOTS+ key pair twice is a security risk, the sender must ensure that they do not pick the same leaf twice to sign a message. This can only be achieved by keeping track of all signatures made with one XMSS key pair, and verifying each time that the leaf chosen has not previously been used. While introducing such state is a valid approach, and XMSS is itself considered quantum resistant [15], it is impractical at best. Thus, SLH-DSA improves upon XMSS by eliminating the need for state.

7.4 FORS

Forest Of Random Subsets (FORS) is a few-time signature scheme, i.e. it allows for the signing of multiple messages using the same key pair. It is built upon Merkle Trees. The generation of the FORS private key is dependant on the length of the message signed, which is among the reasons for why FORS does not sign the actual message, but instead a message digest. This way, it is ensured that every message has the same length [16].

Key Generation To generate the FORS private key, the message length is expressed by a multiple of two integers, $|m| = k \cdot a$. This is used to calculate the parameter $t := 2^a$. The second parameter, n , is obtained as defined in Section 7.2. The FORS private key is then generated using a PRF as k sets containing t n -byte strings. Once again, the private key is a seed for this PRF.

To obtain the public key, each set is used to provide the leaves for a Merkle Tree, creating a forest of k Merkle Trees, each with t leaves. The public key is then obtained as the concatenation of each root $r_{0\dots k-1}$ [17].

Signature Generation and Verification A FORS signature of a message is obtained by a principle similar to XMSS. The message digest is split into k a -bit chunks. The value of the i -th message chunk determines selects a leaf sk_{ij} of the i -th tree according to a pseudo-random procedure. The signature provides every chosen leaf, and the authentication path for its respective tree.

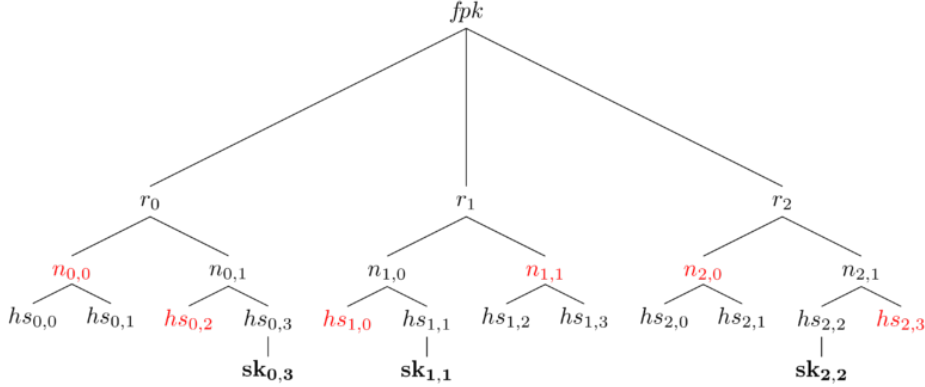


Figure 5: A FORS forest with $a = 2$ and $k = 3$ [17]

The signature is verified by re-computing the root for every one of the k Merkle Trees. The re-computed roots are then concatenated to compute a public key candidate. If this candidate matches the senders' public key, the signature is valid.

7.5 Putting it all together

Now that the underlying schemes have been established, SLH-DSA can be introduced as the combination of these schemes. By combining XMSS with FORS, SLH-DSA is able to sign up to 2^{64} messages with the same key pair.

Key Generation The SLH-DSA private key is the combination of a seed to generate random strings for FORS, and all WOTS+ private keys used in the XMSS Merkle Tree. The SLH-DSA public key is the root of the XMSS Merkle Tree.

Signature Generation and Verification A SLH-DSA signature for a message is obtained by the following steps [16]:

1. Sign the message digest using FORS (see 7.4)
2. Select a leaf wpk_i of the XMSS based on message-dependant pseudo-randomness
3. Sign the FORS public key using XMSS with the selected leaf wpk_i (see 7.3)

The receiver verifies the signature by retracing these steps:

1. Verify the FORS signature (see 7.4)
2. Verify the reconstructed FORS public key's integrity by verifying its XMSS signature (see 7.3)

7.6 Quantum-Resistance

SLH-DSA bases its quantum-resistance primarily on the **Pre-image Resistance** property of Cryptographic Hash Functions. Proving this in detail far exceeds the scope of this contribution, however its idea is motivated here, based on [18]: Even when possessing a quantum computer, an attacker can only try to reverse a Hash Function by brute-forcing it. While quantum computers do offer an advantage over classical computers in this field using Grover's Algorithm, this only offers brute-forcing in $\mathcal{O}(\sqrt{n})$ time, opposed to the $\mathcal{O}(n)$ time that it would take a classical computer. This speedup can be counteracted by doubling the size of the Hash Function's pre-image, which SLH-DSA accounts for by i.a. key size.

8 Conclusion

In this contribution to the class E-Book we have presented why and how quantum computers pose a severe threat to state-of-the art cryptography. We have presented the RSA Algorithm, the currently most wide-spread asymmetric encryption scheme, and how Shor's Algorithm offers an exponential speed-up in breaking it, given a strong enough quantum computer. Although quantum computers currently are only strong enough to factor really small numbers, as shown in our example, the threat of quantum computers becoming strong enough already has implications for the current day, given the practice of Store-Now-Decrypt-Later. In order to ensure safe communication even in a world with strong quantum computers, different cryptographic schemes need to be used, which are not vulnerable to quantum attacks. We presented two of them: ML-KEM and SLH-DSA, both of which are standardized and currently considered secure. As these are intended for wide adoption, they are likely to replace RSA as the standard of encryption not too long in the future.

References

- [1] R. Rivest, A. Shamir and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM* 21.2, 1978.
- [2] C. Paar and J. Pelzl, *Understanding Cryptography*. Springer, 2010.
- [3] D. Warburton and S. Vinberg. F5 Labs. (20th Oct. 2021), [Online]. Available: <https://www.f5.com/labs/articles/threat-intelligence/the-2021-tls-telemetry-report> (visited on 08.05.2025).
- [4] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition. Cambridge University Press, 2010.
- [5] M. Hofmeister, *Quantum Computing verstehen*, 6th ed. Wiesbaden: Springer Vieweg, 2022.
- [6] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700. [Online]. Available: https://cc.ee.ntu.edu.tw/~rbwu/rapid_content/course/QC/Shor1994.pdf.
- [7] C. Gidney and M. Ekerå, “How to factor 2048-bit rsa integers in 8 hours using 20 million noisy qubits,” *Quantum*, vol. 5, p. 433, 2021, Preprint available at <https://arxiv.org/abs/1905.09749>. DOI: 10.22331/q-2021-04-15-433.
- [8] Q. Dang, *18520339/uts-quantum-computing*, original-date: 2024-04-15T05:19:18Z, 2nd May 2025. [Online]. Available: <https://github.com/18520339/uts-quantum-computing> (visited on 09.05.2025).
- [9] Y. Peng, K. Hietala, R. Tao *et al.*, *A formally certified end-to-end implementation of shor’s factorization algorithm*, 2022. DOI: arXiv.2204.07112. arXiv: 2204.07112 [cs.PL].
- [10] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” U.S. Department of Commerce, Federal Information Processing Standard (FIPS) 203, 13th Aug. 2024. DOI: 10.6028/NIST.FIPS.203. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final> (visited on 09.05.2025).
- [11] P. Vorobets, O. Vakhula, A. Horpenyuk and N. Korshun, “Implementing post-quantum kems: Practical challenges and solutions (short paper),” in *CPITS II*, 2024. [Online]. Available: <https://ceur-ws.org/Vol-3826/short9.pdf> (visited on 09.05.2025).
- [12] “NIST releases first 3 finalized post-quantum encryption standards,” *NIST*, 13th Aug. 2024, Last Modified: 2025-02-04T12:30:05:00. [Online]. Available: <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards> (visited on 05.05.2025).

- [13] N. I. o. S. a. Technology, “Digital signature standard (DSS),” U.S. Department of Commerce, Federal Information Processing Standard (FIPS) 186-5, 3rd Feb. 2023, p. 3. DOI: 10.6028/NIST.FIPS.186-5. [Online]. Available: <https://csrc.nist.gov/pubs/fips/186-5/final> (visited on 05.05.2025).
- [14] C. D. Giuseppe. “The mathematics behind PQC: Hash functions,” Telsy. (6th May 2024), [Online]. Available: <https://www.telsy.com/en/the-mathematics-behind-pqc-hash-functions/> (visited on 05.05.2025).
- [15] C. D. Giuseppe. “XMSS: Hash-based stateful digital signature,” Telsy. (1st Jul. 2024), [Online]. Available: <https://www.telsy.com/en/xmss-hash-based-stateful-digital-signature/> (visited on 05.05.2025).
- [16] N. I. o. S. a. Technology, “Stateless hash-based digital signature standard,” U.S. Department of Commerce, Federal Information Processing Standard (FIPS) 205, 13th Aug. 2024. DOI: 10.6028/NIST.FIPS.205. [Online]. Available: <https://csrc.nist.gov/pubs/fips/205/final> (visited on 05.05.2025).
- [17] C. D. Giuseppe. “SPHINCS+: Stateless hash-based digital signature,” Telsy. (26th Aug. 2024), [Online]. Available: <https://www.telsy.com/en/sphincs-stateless-hash-based-digital-signature/> (visited on 05.05.2025).
- [18] J.-P. Aumasson, D. J. Bernstein, W. Beullens *et al.*, “SPHINCS+: Submission to the NIST post-quantum project, v.3.1,” National Institute of Standards and Technology (NIST), Technical Report, 10th Jun. 2022, Version Number: 3.1. [Online]. Available: <https://sphincs.org/data/sphincs+-r3.1-specification.pdf>.